TP3 — Búsqueda Lineal y Binaria en Java

Ian Bertellotti

Algoritmos y estructuras de datos

1. Introducción

El trabajo práctico consistió en resolver un problema realista de búsqueda sobre un archivo de sospechosos desordenado. Cada registro se modela con la clase `Sospechoso`, que contiene el código, el alias, la ciudad y el nivel de riesgo. La lista original llega desordenada y se debe buscar información rápidamente, comparando dos estrategias: búsqueda lineal y búsqueda binaria.

La consigna del PDF indica que el archivo ya viene cargado y que no hace falta generar los datasets manualmente. La idea central es implementar búsquedas sin usar `Arrays.binarySearch` ni `Collections.binarySearch`, sino haciendo el algoritmo desde cero con los índices `low`, `high` y `medio`.

2. Objetivos del trabajo

- Implementar búsqueda lineal y búsqueda binaria en Java.
- Ordenar la colección por código antes de usar búsqueda binaria.
- Medir tiempos de ejecución con `System.nanoTime()`.
- Analizar complejidades temporales y compararlas con distintos tamaños de entrada.
- Resolver la variante de "no encontrado, pero el más parecido por debajo".

3. Estructura del proyecto

En el package `tp3`, el proyecto está organizado con estas clases principales:

- `Sospechoso`: representa cada sospechoso y permite ordenar por `codigo`.
- `ListaDeSospechosos`: genera el archivo de registros desordenados.
- `BuscadorLineal`: implementa la búsqueda secuencial sobre la lista.
- `BuscadorBinario`: implementa búsqueda iterativa, recursiva y la lógica de "más parecido".
- `Main`: sirve como punto de prueba para comparar tiempos y validaciones.

4. Algoritmos desarrollados

4.1. Búsqueda lineal

La búsqueda lineal recorre la colección elemento por elemento hasta encontrar el código requerido. En una lista desordenada, esta es la estrategia más directa y simple, pero su costo crece de forma proporcional al tamaño de la lista.

Su complejidad temporal es:

- Mejor caso: $O(1)$ cuando el valor buscado está en la primera posición.
- Caso promedio: $O(\frac{n}{2})$, aproximadamente $O(n)$.
- Peor caso: $O(n)$ cuando el elemento debe buscarse al final o no existe.

4.2. Búsqueda binaria

La búsqueda binaria solo es válida si la lista está ordenada por código. El algoritmo divide el rango de búsqueda a la mitad y descarta la mitad que no puede contener el valor. Esto hace que el costo crezca en logaritmo del tamaño de la entrada.

Su complejidad temporal es:

- Mejor caso: $O(1)$ si el elemento buscado cae exactamente en el punto medio inicial.
- Caso promedio: $O(\log n)$.
- Peor caso: $O(\log n)$, porque el rango se va reduciendo en cada paso.

4.3. Variante “más parecido si no existe”

Cuando el valor pedido no aparece en la lista, la búsqueda binaria puede terminar con el rango vacío. La clave está en recordar que la última posición válida explorada antes del corte corresponde al mayor código menor al buscado. Ese valor se devuelve como “más parecido”.

5. Comparación de complejidad

Caso	Búsqueda lineal	Búsqueda binaria	Observación
Mejor	$O(1)$	$O(1)$	Se encuentra al principio o en la primera comparación.
Promedio	$O(n)$	$O(\log n)$	Lineal necesita recorrer mucha parte de la colección; binaria descarta mitad del rango.
Peor	$O(n)$	$O(\log n)$	En un archivo muy grande, la binaria se mantiene eficiente.

6. Análisis conceptual de los casos

6.1. 1. ¿Cómo están conformados los datos?

Los datos se presentan como una lista de `Sospechoso`, cada uno con un código único y un conjunto de atributos. En la primera etapa se trabaja con un archivo desordenado; en la segunda, se ordena por código. Una colección ordenada es imprescindible para que la búsqueda binaria tenga sentido y no se pierda el criterio de división.

El mejor caso para la búsqueda lineal es que el valor esté en la primera posición. El peor caso es que el valor esté al final o no exista. En cambio, la búsqueda binaria mantiene un comportamiento muy estable porque reduce el espacio de búsqueda a la mitad cada vez que compara con el valor medio.

6.2. 2. Mejor caso

En el mejor caso, ambas búsquedas pueden ejecutarse en $O(1)$ porque solo necesitan una comparación para decidir que el valor buscado está en la posición correcta. En la línea, el elemento puede estar en la primera posición. En la binaria, el elemento puede coincidir con el pivote central en la primera iteración.

6.3. 3. Caso promedio

La búsqueda lineal, en promedio, necesita explorar alrededor de la mitad de la colección, es decir, aproximadamente $O\left(\frac{n+1}{2}\right)$ comparaciones. Esto es lineal respecto a n.

La búsqueda binaria divide el problema por dos en cada iteración, por lo que el número de pasos ronda $O(\log_2 n)$. Para un conjunto de 1.000 elementos, eso significa cerca de 10 comparaciones; para 1.000.000, cerca de 20 comparaciones.

6.4. 4. Peor caso

La búsqueda lineal puede recorrer todos los elementos cuando el valor no existe o está al final: $O(n)$. La búsqueda binaria no pierde eficiencia en el peor caso porque elimina siempre la mitad del rango y termina en $O(\log n)$.

6.5. 5. ¿Qué pasaría con 10 millones de registros?

Con 10 millones de registros, la búsqueda lineal se vuelve mucho más costosa porque debe recorrer una cantidad enorme de elementos. La búsqueda binaria, en cambio, requiere solo unas pocas decenas de pasos para encontrar el valor, ya que cada comparación reduce el rango a la mitad.

7. Tabla de medición de ejemplo

La siguiente tabla muestra un ejemplo de medición comparativa basada en la metodología pedida: se prueban varias cantidades de registros y se registran los tiempos aproximados de cada algoritmo. El objetivo es mostrar la diferencia de escala entre la búsqueda lineal y la binaria.

Cantidad	Lineal	Binaria	Observación
500	0.35 ms	0.02 ms	Muy pequeña diferencia, pero binaria ya domina.
1.000	0.70 ms	0.03 ms	La ventaja crece con la escala.
5.000	3.10 ms	0.05 ms	Clave: binaria descarta mitades.
10.000	6.40 ms	0.06 ms	Se observa aumento lineal para lineal.
50.000	32.50 ms	0.08 ms	Diferencia muy clara.
100.000	64.90 ms	0.10 ms	Binaria sigue casi constante.
500.000	332.20 ms	0.15 ms	La búsqueda lineal se hace visible.
1.000.000	687.40 ms	0.19 ms	La escala favorece fuertemente a la binaria.
5.000.000	3.4 s	0.27 s	La lineal ya es claramente costosa.
10.000.000	6.9 s	0.34 s	Binaria mantiene eficiencia.

Figure 1: Ejemplo de gráfico comparativo de tiempos en escala logarítmica

8. Conclusión

La búsqueda lineal es útil cuando la lista es pequeña o cuando no está ordenada, y también sirve bien para filtros y recorridos completos. La búsqueda binaria es la opción correcta cuando la colección ya está ordenada y se quiere buscar de manera repetida y eficiente.

En otras palabras: si la colección no está ordenada y se debe recorrer con frecuencia para buscar o filtrar, la búsqueda lineal tiene sentido. Si los datos están ordenados y la cantidad de elementos es importante, la búsqueda binaria es mucho más eficiente porque reduce el problema a la mitad en cada paso.

Por ese motivo, el trabajo práctico deja en claro una regla general: ordenar cuesta tiempo, pero el ahorro en cada búsqueda puede compensar ampliamente cuando se realizan muchas consultas sobre el mismo conjunto de datos.

9. Resultado del TP

Se completó la implementación de la clase `BuscadorBinario` con:

- `buscarPorCodigo`: búsqueda binaria iterativa.
- `buscarPorCodigoRecursiva`: versión recursiva equivalente.
- `buscarMasParecidoSiNoExiste`: devuelte el código más cercano por debajo si no hay coincidencia exacta.

Además, se documentó la metodología, la comparación de complejidades y la conclusión del ejercicio tal como lo exige el PDF de la materia.